D

TAREQO

02 — Architecture

Version 2.1.3+23 · 14 septembre 2026

Table des matières

Résumé 3

La règle de dépendance (le cœur de l'architecture) 3

- Exemple réel : comment un écran atteint les données 3

Pourquoi la Clean Architecture dans ce projet ? 4

Décisions structurelles importantes 4

- Les contrats comme interfaces (DIP) 4
- GetIt comme Service Locator (composition root) 4
- Riverpod + GetIt ensemble 5
- Import conditionnel pour le web (SQLite) 5

Agrégations côté serveur (H2) : l'aperçu du Z-report 5

- Le problème résolu par H2 5
- Le flux en mode Supabase (H2) 6
- Décisions d'architecture 6

Pattern d'erreur : exceptions du domaine + `CrashReporting` 7

Documents connexes 7

Résumé

TAREQO suit la Clean Architecture en trois couches, le tout dans lib/ :

```
lib/
 main.dart # Bootstrap : initialise Supabase + GetIt + ProviderScope
 app.dart # Widget racine (MaterialApp, thème, AuthGate)
 core/ # Préoccupations transversales (n'appartiennent à aucune couche)
  config/ # Configuration (URL Supabase/clé anon via --dart-define)
  database/ # Façade SQLite (import conditionnel io × stub)
  di/ # Service Locator (GetIt) — registre central des dépendances
  errors/ # (vide — les exceptions du domaine remplacent l'ancienne hiérarchie Failure)
  i18n/ # Helpers L10n + chaînes des descriptions de transaction
  observability/ # Signalement de crash (Sentry) — CrashReporting
  network/ # Bootstrap Supabase + câblage du stockage de session sécurisé
  security/ # Stockage sécurisé (Keychain/Keystore) + SecureSupabaseLocalStorage
  theme/ # Thème Material 3 (AppTheme, AppColors)
  utils/ # Helpers génériques (Formatters fr-BR)
 domain/ # CŒUR : règles métier pures, aucune dépendance externe
  entities/ # Entités immuables (Freezed) : Product, Sale, SaleItem, Debt, FinancialTransaction, AppUser
  repositories/ # Contrats abstraits (IProductRepository, ISaleRepository, ...)
  usecases/ # Cas d'usage — orchestrent les règles métier
 data/ # Implémentations concrètes de ce que définit le domaine
  datasources/
   auth/ # AuthRemoteDataSource (Supabase)
   local/ # Contrats + implémentations mémoire et SQLite (product, sale, financial_transaction, debt) + unit of work +
catalogue de forfaits
  models/ # DTO sérialisables (réservé)
  repositories/ # Implémentations des contrats du domaine (ProductRepositoryImpl, ...)
 presentation/ # Tout ce que l'utilisateur voit et avec quoi il interagit
  controllers/ # Providers/notifiers Riverpod (auth, products, sales, finance, debts)
  screens/ # Écrans par module (auth, shell, dashboard, products, sales, financial, credit)
  widgets/ # Widgets partagés (KpiCard, EmptyState, StatusChip, AppSheet, ...)
```

La règle de dépendance (le cœur de l'architecture)

Les dépendances pointent toujours vers l'intérieur :

```
presentation → domain (l'UI ne connaît que des abstractions : entités, cas d'usage, contrats)
data → domain (implémente les contrats du domaine)
core → nothing (sert toutes les couches)
domain → nothing (ne connaît ni Flutter, ni SQLite, ni Riverpod, ni GetIt...)
```

- Le domaine n'importe aucune autre couche. Entités, contrats et cas d'usage sont du Dart pur (avec annotations Freezed pour l'immuabilité et la sérialisation).
- L'inversion se produit au câblage : c'est le ServiceLocator (GetIt) au bootstrap qui décide *quelles* implémentations concrètes alimentent les contrats. Passer de SQLite à la mémoire (ou à une API REST à l'avenir) ne change pas une seule ligne du domaine ni de la présentation.

Exemple réel : comment un écran atteint les données

```
ProductListScreen (widget)
 ↓ ref.watch
```

```
productsProvider (AsyncNotifier)
 ↓ ref.read
getProductsUseCaseProvider → GetProductsUseCase (domain)
 ↓
IProductRepository (contrat du domaine)
 ↓ (implémentation injectée via GetIt)
ProductRepositoryImpl → ProductLocalDataSource (couche data)
 ↓ (SQLite sur natif / en mémoire sur web)
base de données tareqo.db
```

La couche presentation ne connaît que des abstractions — Product, GetProductsUseCase, IProductRepository — jamais de datasources, de SQL ni d'implémentations concrètes.

Pourquoi la Clean Architecture dans ce projet ?

1. Testabilité : les cas d'usage sont testés avec des fakes de contrats — pas de serveur, pas de base de données, rapide et déterministe.
2. Remplaçabilité : la persistance est déjà passée de mémoire → SQLite en changeant un seul enregistrement dans le ServiceLocator ; une future API REST suivrait le même chemin.
3. Maintenabilité : chaque couche a un contrat clair ; les développeurs travaillent dans des modules isolés.
4. Évolutivité d'équipe : les interfaces du domaine servent de « contrats » qui permettent le travail en parallèle (données et présentation contre le même domaine).

Décisions structurelles importantes

Les contrats comme interfaces (DIP)

IProductRepository est une interface abstraite — uniquement des signatures (getAll, getById, add, updateStock). L'implémentation concrète (ProductRepositoryImpl, avec SQLite ou en mémoire) vit dans la couche data. Le domaine dépend de l'abstraction, jamais de l'implémentation.

GetIt comme Service Locator (composition root)

lib/core/di/service_locator.dart enregistre tout dans ServiceLocator.initialize() :

1. SecureStorageService, IAuthRepository et EmployeeAccountService ;
2. Persistance conditionnelle :
3. AppDatabase.openAppDatabase() renvoie true (natif) → enregistre les datasources SQLite + SqliteUnitOfWork ;
4. false (web ou échec de la bibliothèque native) → enregistre les datasources en mémoire + unit of work no-op.
5. Repositories et cas d'usage (identiques dans les deux scénarios, car ils ne dépendent que des contrats).

```
if (await AppDatabase.openAppDatabase()) {
 AppDatabase.registerDatasources(instance);
} else {
 _registerInMemoryDatasources();
}
_registerRepositoriesAndUseCases();
```

Riverpod + GetIt ensemble

GetIt fournit les instances (dépendances concrètes) ; Riverpod définit le graphe d'état réactif. Le pont est fait par de simples providers :

```
final getProductsUseCaseProvider = Provider<GetProductsUseCase>(
 (ref) => ServiceLocator.instance<GetProductsUseCase>(),
);
```

Dans les tests, ces providers sont surchargés avec overrideWithValue/overrideWith — sans toucher à GetIt ni aux datasources.

Import conditionnel pour le web (SQLite)

dart:ffi n'existe pas dans le navigateur. lib/core/database/app_database.dart utilise :

```
export 'app_database_stub.dart' if (dart.library.io) 'app_database_io.dart';
```

- `app_database_io.dart` — ouvre tareqo.db, crée le schéma, ne seed jamais de données (la base démarre vide), expose des transactions réentrantes ;
- `app_database_stub.dart` — openAppDatabase() renvoie toujours false (le web retombe sur la mémoire).

> Attention : Gardez les signatures de `openAppDatabase()` et `registerDatasources()` synchronisées entre les deux fichiers (un commentaire à ce sujet existe dans le code).

Agrégations côté serveur (H2) : l'aperçu du Z-report

Depuis H2 (passage à l'échelle), l'app ne télécharge pas tout l'historique pour calculer les totaux : les agrégations lourdes s'exécutent dans Postgres (RPC SECURITY DEFINER) en UNE seule appel — l'appareil ne reçoit que les chiffres. Ce pattern couvre le flux de trésorerie (get_cash_flow), le rapport de ventes (get_sales_report), les KPI du tableau de bord (get_dashboard_kpis) et l'aperçu de clôture de caisse du jour (Z-report) — get_cash_closing_summary — le sujet de cette section.

Le problème résolu par H2

L'aperçu du Z-report a besoin du résumé du jour du magasin : total des ventes actives, attendu dans le tiroir (ventes en espèces uniquement) et la répartition par moyen de paiement, plus la clôture déjà enregistrée (si elle existe). En mode local, GetCashClosingSummaryUseCase liste toutes les ventes et additionne côté client (aggregateCashClosingDay). Pour les magasins avec un grand historique (10k+ lignes), cela signifie télécharger et traiter tout sur l'appareil à chaque ouverture d'écran — latence et mémoire inutiles.

Le flux en mode Supabase (H2)

```
CashClosingSheet (Financial → Clôture de caisse)
  ref.watch
  ▼
cashClosingSummaryProvider (Riverpod — FutureProvider.autoDispose)
  résout activeStoreId + jour = aujourd'hui
  ▼
GetCashClosingSummaryUseCase (domain)
  serverRepository != null (enregistré uniquement en mode Supabase)
  ▼
SupabaseCashClosingSummaryRepository (data)
  UN SEUL appel RPC — le client ne liste jamais les ventes
  ▼
supabaseClient.rpc('get_cash_closing_summary', {p_store_id, p_day})
  PostgREST → Postgres
  ▼
get_cash_closing_summary (SECURITY DEFINER + garde is_my_store)
  Σ total des ventes ACTIVES du jour (tous les moyens)
  Σ attendu dans le tiroir (uniquement payment_method = 'cash')
  jsonb_object_agg par moyen → répartition (JSONB)
  clôture déjà enregistrée dans cash_closings (si elle existe)
  renvoie JSONB
  ▼
CashClosingSummary (jour, total, attendu, répartition, clôture existante) → UI
```

Décisions d'architecture

- Agréger là où vivent les données : la RPC additionne dans la base (SUM(total_amount) avec status = 'active' sur la fenêtre du jour) — le payload réseau est toujours petit et constant, quel que soit l'historique (passage à l'échelle H2).
- Gardes sur le serveur, pas dans l'UI : SECURITY DEFINER + SET search_path + la garde `is_my_store(p_store_id)` — l'aperçu est lisible par le propriétaire du magasin et les employés ; la *confirmation* (clôture) est managériale (is_manager_of_store dans la RPC close_cash_register). Même un client modifié appelant PostgREST directement est bloqué.
- Le client n'envoie que des paramètres : p_store_id et p_day (début du jour local converti en UTC — même sémantique que close_cash_register, fenêtre [p_day, p_day + 1 day)). Aucune valeur calculée ne quitte l'appareil.
- Mêmes critères, deux endroits : le mode local utilise aggregateCashClosingDay (côté client) avec exactement les mêmes critères que la RPC (ventes actives uniquement ; les annulées restent dehors) — un critère unique partagé avec CloseCashRegisterUseCase, pour que l'aperçu et la confirmation ne divergent jamais.

- Injection facultative (DIP) : GetCashClosingSummaryUseCase reçoit un serverRepository facultatif (instance.isRegistered<ICashClosingSummaryRepository>()). Supabase → SupabaseCashClosingSummaryRepository (RPC) ; local/tests → null (repli côté client). Le domaine et l'UI ne changent pas entre les modes — seul le câblage du ServiceLocator change, comme tout changement de persistance dans ce projet.

> Vérification : sql_check.sh (19.6) et sql_check_live.sh (35/35) valident la RPC contre Postgres 16/17 (SECURITY DEFINER, garde, agrégation côté serveur) ; l'E2E integration_test/cash_closing_summary_live_test.dart prouve le payload réel ({p_store_id, p_day} avec le jour local en UTC) contre le vrai Supabase.

Pattern d'erreur : exceptions du domaine + CrashReporting

L'app n'utilise pas de hiérarchie Failure. Les erreurs métier sont des exceptions du domaine (ex. InvalidSaleException, InsufficientStockException) levées par les cas d'usage et attrapées par les écrans, qui les traduisent en messages conviviaux. Les erreurs inattendues (et toute exception attrapée) sont routées vers `CrashReporting.capture` — debugPrint en debug, Sentry en release quand SENTRY_DSN est présent (voir lib/core/observability/crash_reporting.dart).

Documents connexes

- Domaine — entités, contrats et cas d'usage.
- Couche de données — datasources, SQLite et Unit of Work.
- Présentation — providers, écrans et widgets.
- Runbook de migration de la clôture de caisse — runbook des Parties 25/26 (table cash_closings + RPC close_cash_register/get_cash_closing_summary) appliquées en production.